

NumPy — Complete Study Notes

AI Engineer Learning Path

These notes summarize the NumPy topics covered step-by-step, from arrays to a complete Student Marks Analyzer.

1. What is NumPy?

NumPy (Numerical Python) is a Python library for fast numerical computing. It provides powerful multidimensional arrays and mathematical operations. NumPy is a foundation for data science, machine learning, and AI.

```
import numpy as np

arr = np.array([10, 20, 30, 40])
print(arr)
```

2. Creating Arrays

```
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.array([[1, 2], [3, 4]])

print(a)
print(b)
```

3. Indexing

```
marks = np.array([72, 85, 91, 64, 78])

marks[0]    # first value → 72
marks[-1]   # last value → 78
marks[2]    # third value → 91
```

4. Slicing

```
marks[1:4]   # index 1 through 3
marks[2:5]   # index 2 through 4
marks[::-1]  # reverse array
```

Remember: the stop index is excluded. **arr[start:stop]** includes start but not stop.

5. Array Size

```
marks.size    # number of elements
marks.shape   # dimensions/shape
marks.ndim    # number of dimensions
```

6. Basic Statistics

```
np.mean(marks) # average
np.max(marks)  # highest
np.min(marks)  # lowest
np.sum(marks)  # total
```

7. Finding Positions with argmax / argmin

```
highest_index = np.argmax(marks)
lowest_index = np.argmin(marks)

print(names[highest_index])
```

```
print(marks[highest_index])
```

argmax() and **argmin()** return positions (indexes), not the actual values.

8. Boolean Indexing

```
passed = marks >= 60
```

```
marks[passed]          # passing marks
names[passed]          # names of passing students
```

```
marks[marks < 60]      # failed marks
names[marks < 60]      # failed students
```

Boolean indexing is one of the most important NumPy techniques for filtering data.

9. Counting with Boolean Conditions

```
number_passed = np.sum(marks >= 60)
number_failed = np.sum(marks < 60)
```

10. Vectorized Mathematical Operations

```
marks + 5
marks - 5
marks * 2
marks / 100
```

NumPy applies the operation to every element without writing a Python loop.

11. Broadcasting

```
marks = np.array([72, 85, 91, 64])
bonus = np.array([5, 3, 2, 7])
```

```
new_marks = marks + bonus
```

Broadcasting allows compatible arrays or values to participate in element-by-element operations.

12. Reshape and 2D Arrays

```
marks = np.array([72, 85, 91, 64, 78, 55, 89, 93, 67, 81])
```

```
marks_2d = marks.reshape(2, 5)
```

```
print(marks_2d)
# [[72 85 91 64 78]
#   [55 89 93 67 81]]
```

The number of elements must stay the same. Ten elements can become (2,5), (5,2), (1,10), etc., but not (3,4).

13. 2D Indexing

```
marks_2d[0]          # first row
marks_2d[1]          # second row
marks_2d[0, 2]        # row 0, column 2
marks_2d[:, -1]       # last column
```

14. axis

```
np.mean(marks_2d, axis=0)  # operate down columns
np.mean(marks_2d, axis=1)  # operate across rows

np.sum(marks_2d, axis=0)
np.sum(marks_2d, axis=1)
```

```

np.max(marks_2d, axis=0)
np.max(marks_2d, axis=1)
np.min(marks_2d, axis=0)
np.min(marks_2d, axis=1)

```

Memory trick: axis=0 collapses rows and works down columns. axis=1 collapses columns and works across rows.

15. Random Numbers

```

np.random.randint(1, 101, 10) # 10 random integers from 1-100
np.random.rand(5)             # 5 random values from 0-1
np.random.randint(0, 101, (3, 4)) # 3x4 random array

```

Random values are useful for testing, simulation, experiments, and later for understanding model weight initialization.

16. Sorting

```

np.sort(marks)           # sorted values
np.sort(marks)[::-1]     # descending order

ranking = np.argsort(marks)
names[ranking]
marks[ranking]

# highest → lowest
ranking = np.argsort(marks)[::-1]

```

sort() returns values. **argsort()** returns the indexes that produce the sorted order.

17. Combining Arrays

```

a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.concatenate((a, b))
np.vstack((a, b))
np.hstack((a, b))

```

concatenate = join arrays; vstack = stack vertically/add rows; hstack = join horizontally.

18. Splitting Arrays

```

x = np.array([10, 20, 30, 40, 50, 60])

parts = np.split(x, 3)

print(parts)
# [array([10, 20]), array([30, 40]), array([50, 60])]

```

19. Missing Values: NaN

```

marks = np.array([80, 75, np.nan, 90, 85])

np.isnan(marks)           # identify missing values
np.nanmean(marks)         # mean while ignoring NaN
marks[~np.isnan(marks)]   # values that are not NaN

```

NaN means Not a Number and commonly represents missing numerical data. Handling missing values is essential before many machine-learning workflows.

20. np.where()

```

marks = np.array([45, 60, 75, 50, 90])

result = np.where(marks >= 60, "Pass", "Fail")

```

```
print(result)
```

```
failed_indexes = np.where(marks < 60)
```

Pattern: **np.where(condition, value_if_true, value_if_false)**. It is useful for conditional labels and transformations.

21. Complete Student Marks Analyzer

```
import numpy as np

names = np.array([
    "Rahim", "Karim", "Hasan", "Nadia", "Sadia",
    "Imran", "Tanvir", "Ayesha", "Rafi", "Mim"
])

marks = np.array([
    72, 85, 91, 64, 78,
    55, 89, 93, 67, 81
])

# Basic statistics
average = np.mean(marks)
highest = np.max(marks)
lowest = np.min(marks)

print("Average:", average)
print("Highest:", highest)
print("Lowest:", lowest)

# Highest / lowest students
highest_index = np.argmax(marks)
lowest_index = np.argmin(marks)

print("Highest student:", names[highest_index])
print("Lowest student:", names[lowest_index])

# Pass / fail
result = np.where(marks >= 60, "Pass", "Fail")

print("Results:", result)
print("Passed:", names[marks >= 60])
print("Failed:", names[marks < 60])

# Above average
above_average = marks > average
print("Above average:", names[above_average])

# Counts
print("Passed count:", np.sum(marks >= 60))
print("Failed count:", np.sum(marks < 60))

# Ranking
ranking = np.argsort(marks)[::-1]

print("Ranking:")
print(names[ranking])
print(marks[ranking])

# Top 3
print("Top 3 students:")
print(names[ranking[:3]])
print(marks[ranking[:3]])
```

Project Results

Metric	Result
Class average	77.5
Highest mark	93
Highest student	Ayesha

Lowest mark	55
Lowest student	Imran
Passing threshold	60
Failed student	Imran

22. NumPy Cheat Sheet

Task	Function / Syntax
Create array	<code>np.array([...])</code>
Number of elements	<code>arr.size</code>
Shape	<code>arr.shape</code>
Dimensions	<code>arr.ndim</code>
Average	<code>np.mean(arr)</code>
Maximum	<code>np.max(arr)</code>
Minimum	<code>np.min(arr)</code>
Total	<code>np.sum(arr)</code>
Maximum index	<code>np.argmax(arr)</code>
Minimum index	<code>np.argmin(arr)</code>
Sort values	<code>np.sort(arr)</code>
Sort indexes	<code>np.argsort(arr)</code>
Reshape	<code>arr.reshape(rows, cols)</code>
Random integers	<code>np.random.randint(...)</code>
Join arrays	<code>np.concatenate(...)</code>
Vertical stack	<code>np.vstack(...)</code>
Horizontal stack	<code>np.hstack(...)</code>
Split	<code>np.split(...)</code>
Missing values	<code>np.isnan(arr)</code>
Mean ignoring NaN	<code>np.nanmean(arr)</code>
Conditional selection	<code>arr[condition]</code>
Conditional values	<code>np.where(condition, x, y)</code>

23. What You Should Remember for AI Engineering

You do not need to memorize every NumPy function. Focus on understanding arrays, indexing/slicing, Boolean filtering, vectorized operations, broadcasting, reshaping, axis, sorting, missing values, and conditional operations. You can look up less-common functions when a project requires them.

Next in the roadmap: Pandas → Matplotlib → Seaborn → Data Analysis → scikit-learn → Machine Learning.